Burp Suite

A Practical Guide to Web Application Testing

This guide covers Burp Suite, the standard tool for intercepting, inspecting, and manipulating web traffic during application security testing, for use in authorized environments only — personal labs, PortSwigger Web Security Academy, HackTheBox/TryHackMe, and engagements where you have explicit written permission to test. Testing web applications without authorization is illegal in most jurisdictions.

Contents

- 1. What Burp Suite Does and Why It Matters
- 2. Community vs Professional
- 3. Setup: Proxy and Browser Configuration
- 4. Proxy: Intercepting Traffic
- 5. HTTP History and Target Site Map
- 6. Repeater: Manual Request Manipulation
- 7. Intruder: Automated Attacks
- 8. Decoder and Comparer
- 9. Extensions (BApp Store)
- 10. Scanner (Professional Only)
- 11. Practical Workflow Example
- 12. Good Practice & Legal Notes

1. What Burp Suite Does and Why It Matters

Burp Suite is an intercepting proxy: it sits between your browser and the target web application, letting you see, pause, and edit every request and response as it happens. Where Nmap maps a network and Metasploit exploits a service, Burp is the tool for actually understanding and manipulating how a web application behaves — testing authentication, input validation, session handling, and business logic.

Burp is generally used to answer questions like:

- What requests does the application actually send, including hidden parameters and headers?
- What happens if I change a parameter, cookie, or role value the app assumes I won't touch?
- Can I automate testing many input variations quickly instead of by hand?
- Where does the application trust client-side input that it shouldn't?

2. Community vs Professional

Edition	What You Get
Community (free)	Proxy, Repeater, Intruder (rate-limited), Decoder, Comparer, Sequencer
Professional (paid)	Everything in Community plus the full-speed automated Scanner, unthrottled Intruder, and proje

Community Edition covers the vast majority of manual testing and learning — it's what PortSwigger's Web Security Academy labs are built around, and what most of this guide assumes.

3. Setup: Proxy and Browser Configuration

Burp works by running a local proxy listener (127.0.0.1:8080 by default) that your browser sends traffic through.

- Open Burp, go to **Proxy** → **Options**, and confirm a listener is running on 127.0.0.1:8080.
- Configure your browser to use 127.0.0.1:8080 as its HTTP/HTTPS proxy (or use Burp's built-in Chromium browser under **Proxy** → **Intercept** → **Open Browser**, which is pre-configured).
- To intercept HTTPS without certificate warnings, visit http://burp on the proxied browser and install Burp's CA certificate.

Using Burp's bundled browser avoids most proxy and certificate headaches and is the recommended starting point, especially early on.

4. Proxy: Intercepting Traffic

The Proxy tab is where you watch and control traffic in real time.

Control	Effect
Intercept is on/off	Pauses each request so you can inspect or edit it before it's forwarded
Forward	Sends the currently held request/response on to its destination
Drop	Discards the currently held request instead of sending it
Action → Send to Repeater	Sends the request to Repeater for manual replay and editing
Action → Send to Intruder	Sends the request to Intruder for automated variation testing

Leaving Intercept permanently on gets tedious fast for normal browsing. A common pattern is to turn Intercept on only when you're about to trigger the specific request you want to catch and edit (a login, a password reset, a file upload), and leave it off otherwise while relying on HTTP History to review everything after the fact.

5. HTTP History and Target Site Map

Every request that passes through the proxy, intercepted or not, is logged under **Proxy** → **HTTP History**. This is usually more useful day-to-day than live interception since you can browse the app normally and go back to review anything interesting afterward.

The **Target** → **Site map** tab builds a tree of every URL, parameter, and resource discovered so far as you browse — a passive map of the application's attack surface that grows the more of the app you click through.

Right-click any request in HTTP History or the Site map to send it to Repeater or Intruder, add it to scope, or view it in full detail (headers, body, response, and rendered response).

6. Repeater: Manual Request Manipulation

Repeater lets you take a single captured request, edit any part of it (parameters, headers, cookies, body), and resend it as many times as you like, comparing responses side by side.

- Send a request to Repeater from Proxy or Target (right-click → Send to Repeater).
- Edit any field directly in the request pane — change a parameter value, remove a header, alter a cookie.
- Click Send and review the response pane; use the Pretty/Raw/Hex tabs depending on content type.
- Use tabs to keep several variants of the same request open side by side for comparison.

Repeater is where most manual logic testing happens — trying to access another user's data by changing an ID, testing whether a role check is enforced server-side, or probing how an input field handles unexpected characters.

7. Intruder: Automated Attacks

Intruder automates sending the same request repeatedly with different payloads substituted into marked positions — useful for brute-forcing, fuzzing parameters, or enumerating valid values.

Attack Type	Behavior
Sniper	One payload set, cycled through one position at a time
Battering ram	One payload set, same value inserted into all positions simultaneously
Pitchfork	Multiple payload sets, iterated in parallel across multiple positions
Cluster bomb	Multiple payload sets, every combination of positions tried

```
1. Send a request to Intruder
2. Mark injection points with the section-sign (§) buttons, e.g. §username§
3. Choose an attack type (Positions tab)
4. Add a payload list (Payloads tab) - simple list, numbers, or a wordlist
5. Start attack and review response length/status codes for anomalies
```

Community Edition throttles Intruder's request rate, which makes it noticeably slower than Professional for large wordlists — fine for learning, more limiting for large-scale fuzzing.

8. Decoder and Comparer

Decoder encodes and decodes data in common formats (URL, HTML, Base64, hex, and more) and can chain operations — handy for reading an encoded token or crafting an encoded payload by hand.

Comparer does a word- or byte-level diff between two requests, responses, or arbitrary pieces of text — useful for spotting exactly what changed between two similar responses (e.g. a valid vs invalid login) when the difference isn't obvious by eye.

9. Extensions (BApp Store)

The **Extensions** tab connects to the BApp Store, a marketplace of community-built add-ons that extend Burp's functionality. Popular ones include:

- **Logger++** — enhanced request/response logging with filtering
- **Autorize** — automated authorization testing across user roles
- **JSON Web Tokens** — inspect and edit JWTs directly in the request
- **Param Miner** — discovers hidden, unlinked parameters a target might process

10. Scanner (Professional Only)

Burp's Scanner (Professional Edition) automatically crawls an application and tests for common vulnerability classes — SQL injection, XSS, and dozens of others — generating a findings report with confidence ratings.

It's a strong starting point for coverage on a large application, but it doesn't replace manual testing for business-logic flaws, broken access control, or anything that requires understanding what the application is supposed to do rather than just how it responds to malformed input.

11. Practical Workflow Example

A realistic sequence for testing a single authorized lab target (e.g. a PortSwigger Academy lab):

- **Set up:** open Burp's built-in browser and confirm the proxy listener is running
- **Browse normally:** click through the app's core functionality once with Intercept off, building up HTTP History and the Site map
- **Review:** read through HTTP History for interesting endpoints — login, password reset, file upload, anything with an ID or role parameter
- **Manual testing:** send suspicious requests to Repeater and try modifying parameters, cookies, and headers to see how the server responds
- **Automate where useful:** send repetitive checks (e.g. testing a range of IDs) to Intruder with an appropriate attack type and payload set
- **Confirm and document:** once a flaw is confirmed in Repeater, note the exact request/response pair that demonstrates it

12. Good Practice & Legal Notes

- Only test applications you own or have explicit written authorization to test (a signed scope-of-work, PortSwigger Academy labs, HTB/TryHackMe, or your own local apps).
- Testing production web applications without authorization is illegal in most jurisdictions, even if the flaw found was minor or unintentional.
- Be careful with Intruder and Scanner against real targets — high request volumes can trip rate limits, lock accounts, or degrade service for other users.
- Keep scope tight (Target → Scope) so accidental requests to out-of-scope domains don't happen while testing.
- Save Burp projects (Professional) or export key requests/responses so findings can be reviewed and reported later without re-discovering them.

This guide is meant as a working reference alongside hands-on labs (PortSwigger Web Security Academy, HTB, TryHackMe) and doesn't replace understanding the web technologies being tested.